

AI/ML Technical Assignment Report

Customer Support Ticket Classification System

Customer Support Ticket Classification

*Text Classification, Semantic Embeddings, Explainability and Automated
Response Suggestion*

Submitted by

Tuhin Kumar Singha Roy

AI/ML Fresher Technical Assignment

September 2026

Table of Contents

List of Figures	II
List of Tables	III
Abstract	IV
Chapter 1: Introduction and Problem Understanding	1
1.1 Problem Context	1
1.2 Project Objective	1
1.3 Scope	2
1.4 System Flow	2
Chapter 2: Dataset and Data Preprocessing	3
2.1 Dataset Source	3
2.2 Dataset Characteristics	3
2.3 Synthetic Data Design	3
2.4 Missing Values	4
2.5 Duplicate Handling	4
2.6 Text Preprocessing	4
Chapter 3: Exploratory Data Analysis	5
3.1 Purpose of EDA	5
3.2 Category Distribution	5
3.3 Priority Distribution	6
3.4 Observations	6
Chapter 4: Machine Learning Model Development	7
4.1 Model Selection	7
4.2 Feature Extraction with TF-IDF	7
4.3 Training and Testing Split	7
4.4 Prediction Mechanism	8
4.5 Embedding-Based Extension	8
4.6 Explainability	8
Chapter 5: Training Results, Predictions, and Application	9
5.1 Evaluation Results	9
5.2 Interpretation of Results	10
5.3 Unseen Ticket Predictions	10
5.4 Web Application	11

5.5	Semantic Similarity Extension	11
5.6	What the Semantic Component Adds	12
5.7	Semantic Similarity Example	12
5.8	Secondary Model Evaluation	13
5.9	Explainability	14
5.10	Bulk Ticket Classification	14
5.11	Bonus: Automatic Response Suggestion	15
5.12	Final Comparison and Contribution	15
Chapter 6: Limitations, Future Improvements, and Conclusion		17
6.1	Limitations	17
6.2	Future Improvements	18
6.3	Conclusion	18

List of Figures

3.1	Category distribution in the submitted ticket dataset	5
3.2	Priority distribution in the submitted ticket dataset	6
5.1	Confusion matrix for the primary TF-IDF classifier	10
5.2	Confusion matrix for the secondary semantic classifier	14

List of Tables

2.1	Dataset fields	3
5.1	Primary TF-IDF model evaluation results	9
5.2	Task 5 unseen-ticket predictions	11
5.3	Secondary semantic model evaluation results	13
5.4	Comparison of the two classification approaches	15

Abstract

This project develops a basic machine-learning system for automatically classifying customer support tickets from their description text. A self-generated synthetic dataset of 186 raw tickets across five categories was prepared, cleaned, explored, and split into training and test data. The primary classifier uses TF-IDF features with Logistic Regression. On the submitted 36-ticket test split, the model achieved 94.4% accuracy, 95.1% weighted precision, 94.4% weighted recall, and 94.2% weighted F1 score. A second, embedding-based classifier was also included as an extension, together with explainability features and bulk CSV prediction. Five additional ticket descriptions were tested independently; four of the five matched their intended categories. The project also includes a simple Flask web interface and a rule-based response suggestion feature. The results demonstrate a functioning end-to-end classification pipeline while highlighting the limitations of a small synthetic dataset and the need for larger, more diverse, real-world labeled data before production use.

Chapter 1

Introduction and Problem Understanding

1.1 Problem Context

A company receives customer support requests through email, chat, and other channels. Each request must be read and assigned to a category before it can be routed to the appropriate team. Manual classification can be slow and inconsistent as ticket volume grows.

The objective of this project is to build a basic AI/ML-based classification system that predicts a support-ticket category directly from its description text. The system is intended as a first-pass automation layer rather than a production-ready support platform.

1.2 Project Objective

The project follows the assignment workflow:

1. Prepare a labeled ticket dataset.
2. Inspect and clean the data.
3. Perform exploratory analysis.
4. Convert ticket text into numerical features.
5. Train and evaluate a machine-learning classifier.
6. Test the system on unseen ticket descriptions.
7. Provide a simple user interface.
8. Document the implementation, limitations, and future improvements.

1.3 Scope

The classifier predicts one of five categories: Login Issue, Application Error, Report, Account Update, and Performance. The implementation also contains optional extensions for semantic embeddings, explainability, bulk CSV prediction, and rule-based response suggestions.

1.4 System Flow

The primary processing flow is:

Ticket Description → Text Cleaning → TF-IDF → Logistic Regression → Predicted Category

The project deliberately keeps the primary model simple and explainable because the assignment is designed for an AI/ML fresher and the dataset is small and synthetic.

Chapter 2

Dataset and Data Preprocessing

2.1 Dataset Source

The project uses a self-generated synthetic dataset because a suitable real-world labeled support-ticket dataset was not available for the assignment. The data-generation logic is implemented in `src/generate_dataset.py`.

2.2 Dataset Characteristics

The submitted dataset contains 186 raw tickets and 178 tickets after cleaning. Five categories are represented. The dataset fields are shown in Table 2.1.

Table 2.1: Dataset fields

Field	Description
<code>ticket_id</code>	Unique ticket identifier
<code>customer_name</code>	Customer name
<code>ticket_description</code>	Text used for classification
<code>date</code>	Ticket date
<code>priority</code>	Ticket priority
<code>status</code>	Ticket status
<code>category</code>	Target classification label

The five categories are Login Issue, Application Error, Report, Account Update, and Performance.

2.3 Synthetic Data Design

Descriptions were produced from a limited collection of template sentences per category with random greetings and closing phrases. The generator targets about 15% ambiguous or overlapping tickets. In the submitted dataset, 20 of the 186 raw tickets (approximately 10.8%) matched

the deliberately ambiguous templates. These examples intentionally borrow vocabulary associated with another category, making the classification problem less trivial.

The generator also injects duplicate rows and a small number of missing values so that the preprocessing workflow has actual data-quality issues to handle.

2.4 Missing Values

Rows missing `ticket_description` or `category` were dropped because both are required for supervised training. Missing `priority` and `status` values were filled with `Unknown`; these fields are retained for analysis but are not used as features by the classifier.

2.5 Duplicate Handling

Exact duplicate rows were removed using the Pandas `drop_duplicates()` operation.

2.6 Text Preprocessing

Each ticket description was transformed into a cleaned text field by:

1. converting text to lowercase,
2. removing punctuation and non-alphanumeric characters, and
3. collapsing repeated whitespace.

These transformations reduce superficial variation. For example, “Login” and “login,” are treated consistently instead of becoming separate text tokens.

Chapter 3

Exploratory Data Analysis

3.1 Purpose of EDA

Exploratory analysis was used to understand the number of tickets, category balance, priority distribution, and status distribution before model training. The project includes two primary charts required by the assignment.

3.2 Category Distribution

Figure 3.1 shows the distribution of tickets across the five target categories. This provides a quick check of the relative representation of each class and helps interpret classification performance.

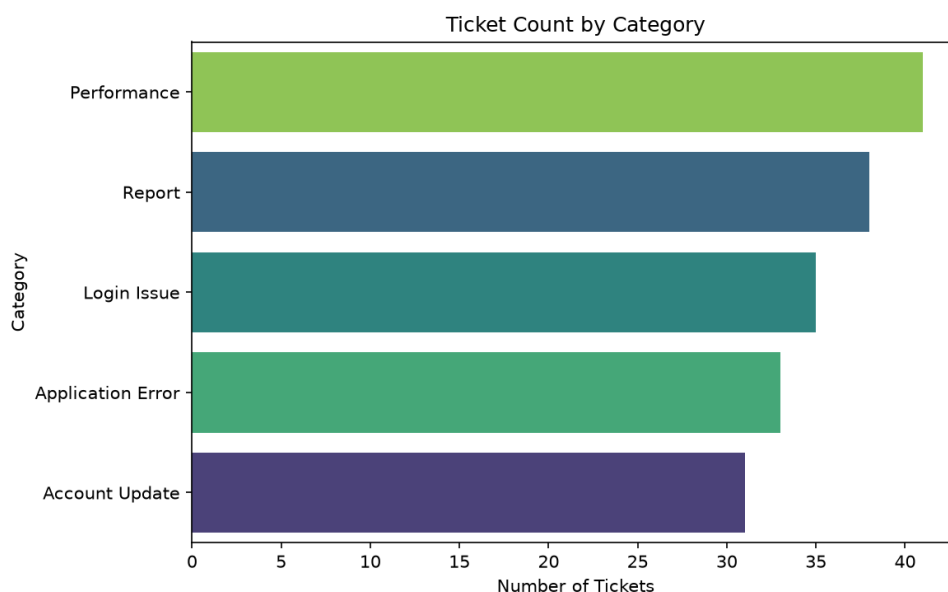


Figure 3.1: Category distribution in the submitted ticket dataset

3.3 Priority Distribution

Figure 3.2 shows the distribution of ticket priorities. Priority is not used as a direct model feature in the main classifier, but it is part of the assignment dataset and is useful for describing the data.

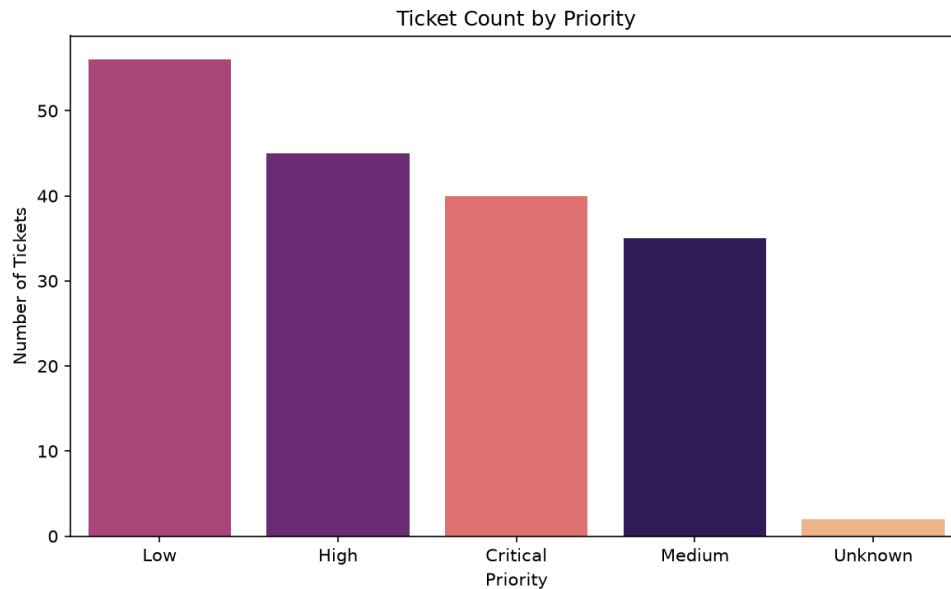


Figure 3.2: Priority distribution in the submitted ticket dataset

3.4 Observations

The EDA confirms that the dataset contains multiple support-ticket categories and different priority levels. Because the dataset is synthetic and template-based, category vocabulary is comparatively structured. This helps explain the strong performance of a conventional TF-IDF classifier while also limiting how directly the score can be generalized to real customer-support traffic.

Chapter 4

Machine Learning Model Development

4.1 Model Selection

The primary classifier is Logistic Regression implemented with scikit-learn. It was selected because it is fast to train, works well with sparse text representations, is straightforward to explain, and provides probability outputs through `predict_proba()`.

The project did not run a benchmark comparison against Naive Bayes or Random Forest. Those models are reasonable alternatives for this type of task, but Logistic Regression was selected directly as the main baseline rather than after a documented model-comparison experiment.

4.2 Feature Extraction with TF-IDF

Ticket descriptions are converted into numerical vectors with `TfidfVectorizer`. The implementation uses unigrams and bigrams with a maximum of 3000 features. TF-IDF reduces the influence of words that occur broadly across tickets while emphasizing terms that are more distinctive for a category.

The training vectorizer is fit only on the training text, and the held-out test text is transformed using the already fitted vectorizer. This avoids fitting the feature representation on test data.

4.3 Training and Testing Split

The cleaned data was divided into 80% training and 20% testing data, giving 142 training tickets and 36 test tickets. The split is stratified by category so that the class proportions remain represented in both subsets.

4.4 Prediction Mechanism

The trained Logistic Regression model produces a category label from the TF-IDF representation of a new description. Probability estimates from `predict_proba()` are displayed as confidence-style scores in the prediction interface. These are model probability estimates; the project does not contain a separate probability-calibration evaluation.

4.5 Embedding-Based Extension

A second classifier using the pretrained `all-MiniLM-L6-v2` sentence-transformer is included as an extension. It is intended to capture semantic similarity beyond exact word overlap. On the submitted 36-ticket test split, the embedding-based and classic systems both report 94.4% accuracy, so the project does not demonstrate an accuracy advantage for the embedding model on this dataset.

4.6 Explainability

The classic model can highlight influential words using learned TF-IDF coefficients. The embedding-based extension instead provides similar past tickets based on embedding similarity, because individual embedding dimensions do not have the same direct word-level interpretation as TF-IDF coefficients.

Chapter 5

Training Results, Predictions, and Application

5.1 Evaluation Results

The primary classification pipeline uses TF-IDF features with Logistic Regression. The model was evaluated on a stratified held-out test set containing 36 tickets. The saved evaluation results are presented in Table 5.1.

Table 5.1: Primary TF-IDF model evaluation results

Metric	Score
Accuracy	94.4%
Weighted Precision	95.1%
Weighted Recall	94.4%
Weighted F1 Score	94.2%

The model correctly classified 34 of the 36 held-out tickets. All Login Issue, Performance, and Report test tickets were classified correctly. The two errors were Application Error tickets: one was predicted as Account Update and the other as Performance. One of these was an intentionally ambiguous example, while the second was a standard Application Error example concerning the reports tab whose wording overlapped with performance-related language.

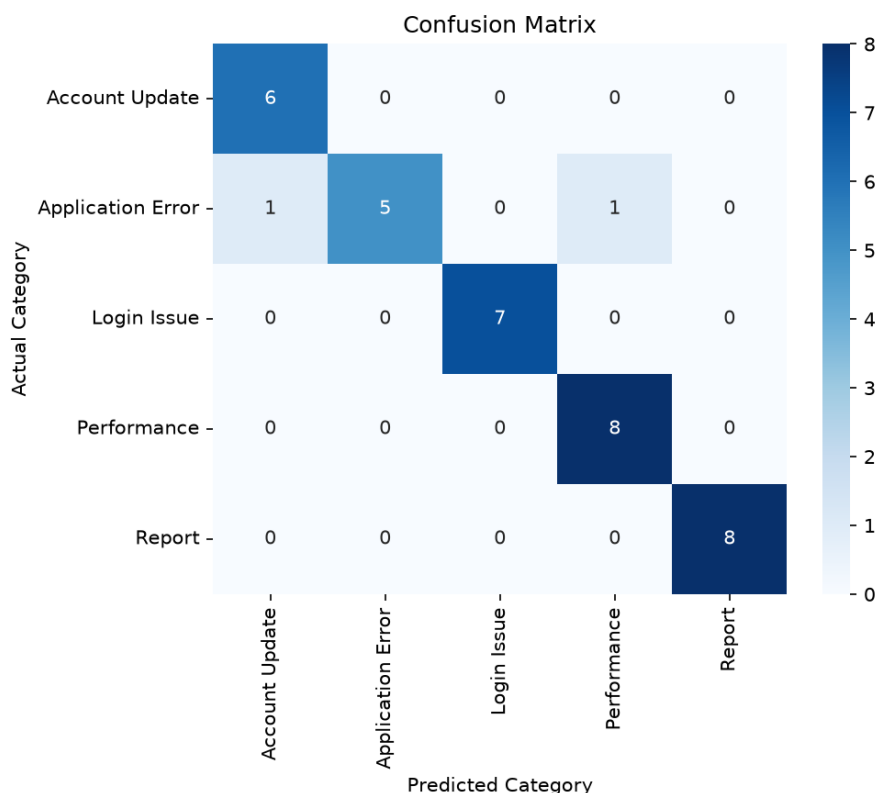


Figure 5.1: Confusion matrix for the primary TF-IDF classifier

5.2 Interpretation of Results

An accuracy of 94.4% means that the classifier correctly identified 34 of the 36 tickets in the held-out test set. The result demonstrates that the selected pipeline can learn useful category patterns from the available data. However, the dataset is small, synthetic, and based on a limited number of templates. Therefore, the reported score should not be interpreted as a 94.4% estimate of real-world customer-support performance.

5.3 Unseen Ticket Predictions

Five additional ticket descriptions were tested using the prediction workflow. These descriptions were not used during model training. Four of the five test cases matched their intended categories. Test case 2, describing repeated application crashes while uploading a document, was predicted as Performance rather than Application Error. Its wording overlaps with performance-related examples in the training data.

Table 5.2: Task 5 unseen-ticket predictions

No.	Ticket description	Prediction	Confidence
1	I can't sign into my account, it keeps saying wrong password even though I reset it.	Login Issue	32.2%
2	The app crashed twice today while I was uploading a document.	Performance	31.9%
3	My invoice report has the wrong total, can you check it?	Report	22.9%
4	I would like to change the email address associated with my account.	Account Update	32.1%
5	Everything on the website loads so slowly today, is there an outage?	Performance	32.8%

The displayed values are probability estimates returned by the Logistic Regression model. The project does not contain a separate probability-calibration evaluation, so they are not presented as calibrated confidence guarantees.

5.4 Web Application

A Flask web application was implemented as the main interaction layer. The application accepts a ticket description and returns the predicted category, probability breakdown, an explanation, and a suggested response. It also provides a Classic/Smart model toggle and supports batch classification from a CSV file.

5.5 Semantic Similarity Extension

A second modelling component was implemented using the pretrained sentence-transformer `all-MiniLM-L6-v2`. In the submitted code, this component is implemented in `src/embeddings_utils.py` and `src/train_embeddings.py`.

The important distinction is that the embedding component is not presented as a demonstrated improvement over the TF-IDF classifier. Its main purpose is to add a semantic representation and an example-based explanation mechanism to the application.

The process is:

Ticket description $\rightarrow$ `all-MiniLM-L6-v2` $\rightarrow$ 384-dimensional sentence embedding $\rightarrow$
Logistic Regression prediction

The same train/test split methodology is used for this secondary model. The embedding model is trained on the training embeddings, while the held-out test embeddings are used only for evaluation. The project also stores the embeddings, text, and category labels of the **training tickets only** in `model/embedding_index.joblib` for later similarity retrieval.

5.6 What the Semantic Component Adds

The semantic component adds functionality that is different from ordinary TF-IDF keyword matching.

1. **Semantic representation.** Each ticket is converted into a dense 384-dimensional vector by `all-MiniLM-L6-v2`. This provides a sentence-level representation rather than a sparse vector consisting of individual word and phrase weights.
2. **Similarity-based retrieval.** The stored training embeddings are searched using cosine similarity. For a Smart-mode prediction, the application retrieves the three most similar historical training tickets. This functionality is implemented by `explain_embeddings()` in `src/explain.py` and is called from `classify()` in `src/predict.py`.
3. **Example-based explanation.** TF-IDF explanations can show influential words because the model operates on human-readable terms. An embedding vector does not provide that same direct word-level interpretation. The project therefore explains a Smart-mode prediction by showing similar previously classified tickets and their categories.
4. **Alternative model mode.** The Flask application exposes a Classic/Smart toggle. Classic uses TF-IDF + Logistic Regression, while Smart uses the sentence-embedding model + Logistic Regression and the similarity-based explanation.
5. **Batch compatibility.** The Smart model can also be selected for bulk CSV classification through the application's batch endpoint, allowing the semantic pipeline to be applied to multiple tickets.

The code therefore uses embeddings for a concrete additional purpose: not just another accuracy score, but semantic similarity, historical-ticket retrieval, and example-based explanation.

5.7 Semantic Similarity Example

The similarity index generated by the project provides concrete evidence that this retrieval layer is operational. For example, within the saved training index, the following two tickets from different categories have a cosine similarity of approximately 74.9%:

Report: “Hello, Report export is stuck at 0 percent and never finishes. This is urgent.”

Application Error: “The export button for my report is broken and does nothing. This is urgent.”

This example does not demonstrate that embeddings improve classification accuracy. Instead, it demonstrates why semantic retrieval can be useful: the system can surface historical tickets that are meaningfully related to the new ticket even when those tickets belong to different categories. That information can help a support agent understand the context surrounding an automated prediction.

The example also exposes an important practical limitation. High similarity between tickets from different categories shows that semantic similarity alone is not sufficient to determine the final support category. Category labels and the trained classifier are still required.

5.8 Secondary Model Evaluation

The embedding-based Logistic Regression classifier was evaluated on the same 36-ticket held-out test set. Its saved metrics are identical to the primary model:

Table 5.3: Secondary semantic model evaluation results

Metric	Score
Accuracy	94.4%
Weighted Precision	95.1%
Weighted Recall	94.4%
Weighted F1 Score	94.2%

The matching results mean that the current experiment does not demonstrate a classification-performance improvement from replacing TF-IDF with sentence embeddings. Consequently, the semantic component is treated as a complementary extension rather than a more accurate replacement for the primary classifier.

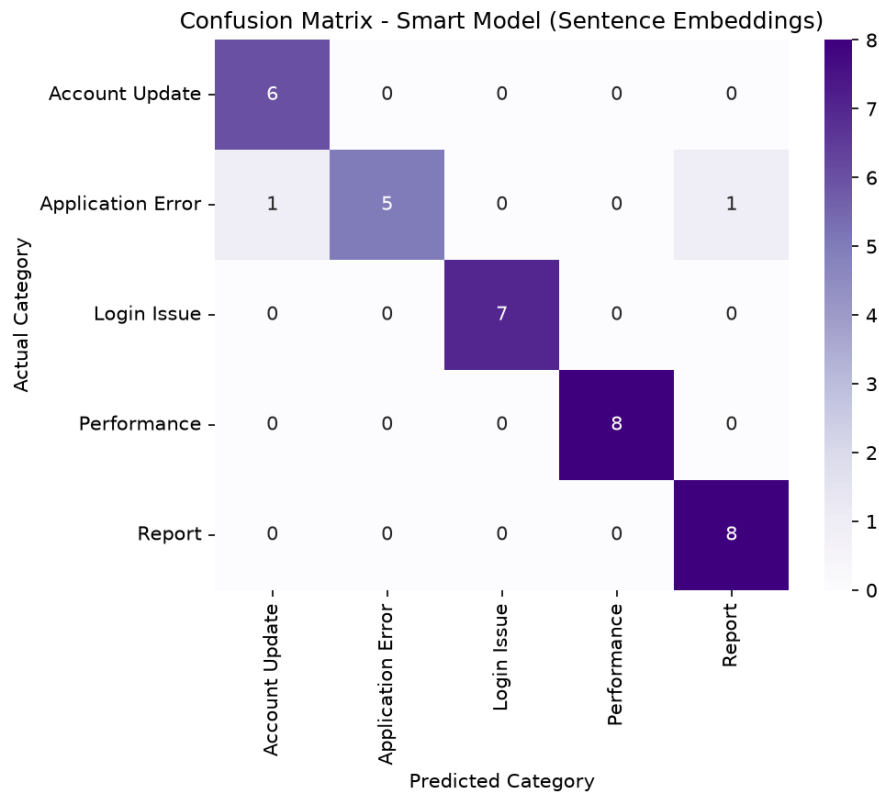


Figure 5.2: Confusion matrix for the secondary semantic classifier

5.9 Explainability

The project implements two model-specific explanation strategies.

For the Classic TF-IDF model, the system examines the model coefficients and identifies the highest-contributing terms present in the ticket. The Flask application highlights these terms in the original ticket text.

For the Smart semantic model, the system does not attempt to assign a human meaning to individual embedding dimensions. Instead, it retrieves the three most similar training tickets using cosine similarity. The returned items contain their ticket text, category, and similarity percentage.

This produces two complementary explanation styles:

- **Keyword-based explanation** for the TF-IDF model.
- **Example-based semantic explanation** for the embedding model.

5.10 Bulk Ticket Classification

The application supports batch classification through CSV upload. The uploaded file is read by the Flask backend, the ticket-description column is identified, and the selected Classic or

Smart model is applied to the batch. The resulting predicted category and probability estimate are returned and can be downloaded as a CSV file.

5.11 Bonus: Automatic Response Suggestion

A substantial extension beyond the core assignment was implemented as the automatic response-suggestion feature. The implementation is a rule-based system defined by the `SUGGESTED_RESPONSES` dictionary in `src/predict.py`.

After the category is predicted, the application selects the corresponding response template and returns it together with the prediction. The same response is exposed through the command-line prediction workflow and the Flask application.

A rule-based approach was deliberately used instead of an external LLM API so that the feature remains self-contained and does not require API keys, credentials, or external inference services.

The generated messages are written as generic support drafts. They are not presented as verified statements about a customer’s account or about the company’s current operational state.

For example:

Input: I forgot my password and cannot login.

Predicted Category: Login Issue

Suggested Response: Please use the “Forgot Password” option on the login page to reset your password. If the problem continues, please contact the support team.

This turns the classifier into a simple agent-assistance workflow: a ticket is classified first, and a category-specific response draft is then generated for human review.

5.12 Final Comparison and Contribution

The project therefore contains a primary lexical classifier and a secondary semantic extension, but their roles are different.

Table 5.4: Comparison of the two classification approaches

Capability	Classic	Smart
Feature representation	TF-IDF	Sentence embedding
Classifier	Logistic Regression	Logistic Regression
Classification accuracy	94.4%	94.4%
Weighted F1	94.2%	94.2%
Keyword explanation	Yes	No
Similar-ticket explanation	No	Yes
Top-3 semantic retrieval	No	Yes
Bulk classification	Yes	Yes

The table shows the actual contribution of the semantic extension. The current experiment provides no evidence that embeddings improve the classification metrics, but they add a different representation and a concrete similarity- based explanation capability that is not available from the TF-IDF model alone. Together with bulk classification and the automatic response-suggestion feature, these extensions move the implementation beyond a basic textbook classifier toward a small support-ticket assistance prototype.

Chapter 6

Limitations, Future Improvements, and Conclusion

6.1 Limitations

1. **Small synthetic dataset:** The submitted dataset contains approximately 180 cleaned tickets generated from a limited set of templates. It demonstrates the pipeline but is much smaller and less varied than real support-ticket traffic.
2. **Limited categories:** Only five categories are supported. A production system would likely require additional categories such as billing, security, feature requests, and other domain-specific issues.
3. **Overlapping vocabulary:** Application Error and Performance examples can share words such as “crash,” “error,” and “slow,” contributing to the observed misclassifications.
4. **Simple model:** TF-IDF with Logistic Regression does not directly model deeper context, word order, or semantic equivalence for unseen phrasing.
5. **Limited language normalization:** No dedicated spelling correction or slang normalization is implemented, which may reduce robustness on informal support messages.
6. **Reproducibility note:** The serialized scikit-learn model artifacts were created with scikit-learn 1.9.1, while an inspection environment used 1.8.0 and produced an `InconsistentVersionWarning`. The project dependency file does not pin the scikit-learn version.
7. **No automated unit-test suite:** The submitted repository does not contain a separate pytest/unittest suite. Functional verification is provided through the project workflow and prediction smoke tests.

6.2 Future Improvements

For a real company deployment, the system could be improved by:

1. collecting a much larger real, anonymized, human-labeled support-ticket dataset;
2. expanding the category taxonomy and monitoring class balance;
3. adding spelling correction, stop-word handling, and other domain-appropriate text normalization;
4. benchmarking alternative classifiers and transformer-based approaches with documented validation experiments;
5. using a low-confidence fallback rather than forcing a prediction when the model is uncertain;
6. continuously retraining from newly labeled tickets and agent feedback;
7. exposing the model through a production API with logging, monitoring, authentication, and database integration; and
8. pinning dependency versions to improve reproducibility.

6.3 Conclusion

This project provides a complete first-pass customer-support ticket classification workflow: data generation, preprocessing, exploratory analysis, feature extraction, model training, evaluation, prediction, a simple web interface, and optional extensions. The primary TF-IDF + Logistic Regression model achieved 94.4% accuracy on the submitted held-out test split, while four of five additional test cases matched their intended categories. The results support the feasibility of the pipeline on the supplied synthetic dataset, while the dataset limitations and lack of real-world validation prevent the reported score from being treated as production performance.